Cinephix — Master System Report & User Manual

2026-09-20 · @Someone

1. System Overview & Technical Architecture

Cinephix is a social film-and-series network: people rate and review what they watch, build lists, post to a shared feed, chat in communities and direct messages, and get compatibility scores against other viewers. It ships as two clients over one API - a React Native app for members, and a web dashboard for moderators.

1.1 The three parts

Part	Stack	Who uses it	Entry point
Mobile app	Expo 57, React Native 0.86 (Fabric), React 19.2, Expo Router 57	Members	`mobile/app`
API + realtime	Node 24, Express 5, Mongoose 8, Socket.io 4.8	Both clients	`backend/server.js`
Web site + admin	Vite 7, React 19, Tailwind 3 + daisyUI, React Router 7	Visitors, moderators	`frontend/src`

The web app was retired as a consumer client. It serves four public routes (landing, about, privacy, terms) plus the moderator dashboard at `/admin`; every retired route redirects to `/`, so old bookmarks never hit a dead page.

[Insert Screenshot: the mobile app home screen beside the web admin dashboard]

1.2 Request lifecycle

```
flowchart LR
  A[Client] --> B[Rate limiter]
  B --> C[Block filter]
  C --> D[Auth]
  D --> E[Controller]
  E --> F[(MongoDB Atlas)]
  E --> G[Response]
  G --> C
```

Every `/api/` request passes a rate limiter, then the global block filter, then route-level authentication. The block filter wraps the outgoing response, which is what makes blocking work everywhere at once (section 8). Authentication reads a JWT bearer token, loads the user, and attaches both the user and their block sets to the request.

1.3 Performance decisions worth knowing

- **Atlas latency is the budget.** Measured against this cluster, every database round trip costs roughly 300 ms, cross-region rather than query-plan. The architecture is shaped around avoiding trips, not around tuning queries.
- **Auth caches the user for 3 seconds** and coalesces concurrent lookups of the same id into one in-flight promise. A tab mount firing 5-9 parallel requests pays one round trip instead of nine.
- **Block sets are cached 60 seconds** per user and dropped for both sides the moment a block changes.
- **Clients cache stale-while-revalidate.** Screens render what they already have on the first frame and refresh underneath. Per-account stores are cleared at sign-out so a second account never sees the first one's data.

1.4 Realtime

Socket.io runs on the same HTTP server, authenticated by the same JWT. It carries chat messages, room join/leave, inbox updates and presence. Push notifications go out through Expo's service in the background, so a slow push never delays the request that triggered it.

1.5 Database schema highlights

MongoDB Atlas, database `cleaned_db`, 22 collections. The ones that carry the product:

Collection	Holds	Notable fields
`users`	Accounts and most personal state	`watchlist`, `toWatchList`, `favoriteMovies/Series`, `movieCineGraph`, `seriesCineGraph`, `blockedUsers`, `pushTokens`, `accountTier`, `isOfficial`, `isAdmin`
`shares`	Feed posts of every type	`type`, `contentData`, `listData`, `pollData`, `badgeData`, `likes`, `viewCount`, `isOfficialPost`

Collection	Holds	Notable fields
`ratings`, `reviews`	Scores and written reviews	`contentId`, `contentType`, `value` / `text`, `likes`
`customlists`	User-made lists	`owner`, `items[]`, `isPublic`, `order`, `likes`
`groups`, `conversations`, `messages`	Communities, DMs, chat	`members[]`, `participants[]`, `roomType`, rich payloads per message type
`footprintevents`	The activity timeline	`type`, `user`, `targetUser`, content refs
`reports`, `tickets`	Moderation and support	see sections 8-9
`movies`, `series`	Local catalogue mirrored from TMDB	`title` / `name`, `poster_path`, genres

Two schema notes that explain a lot of behaviour. First, a user document carries their watched list, watchlist, favourites and per-genre CineGraph maps inline rather than in separate collections, so a profile loads in one read. Second, `blockedUsers` lives on the blocker, and the effect is mutual: the filter treats "people I blocked" and "people who blocked me" as one hidden set.

1.6 Environments

Environment	Backend	Web	Notes
Local	`cd backend && npm run dev` (port 5000)	`cd frontend && npm run dev` (port 5173)	Web dev server proxies `/api` to port 5000, so there is no CORS setup
Mobile dev	Expo dev server, port 8081	-	`mobile/scripts/start-codespace.sh` for tunnelled devices
Production	Render	Render static site	`FRONTEND_URL` drives the CORS allow-list; production builds ship no source maps

2. User Authentication & Onboarding

An account is email plus password, or Google. Email accounts must verify a code before they can sign in - registration deliberately returns no token. Sessions are 30-day JWTs.

2.1 The screens

Screen	File	Purpose
Login	`app/(auth)/login.tsx`	Email + password, link to Google
Register	`app/(auth)/register.tsx`	Username, email, password
Verify email	`app/(auth)/verify-email.tsx`	Emailed code, with resend
Forgot password	`app/(auth)/forgot-password.tsx`	Sends a reset code
Reset password	`app/(auth)/reset-password.tsx`	Code plus new password
Setup profile	`app/setup-profile.tsx`	Photo, username, bio - skippable
OAuth landing	`app/oauth-success.tsx`	Receives the Google redirect

[Insert Screenshot: login, register and verify-email screens side by side]

2.2 Registration flow

```
flowchart TD
  A[Register] --> B[Account created<br/>unverified]
  B --> C[Code emailed]
  C --> D[Verify code]
  D --> E[Token issued]
  E --> F[Setup profile]
  F --> G[Home]
```

Server-side validation: username 3-20 characters, a well-formed email, password at least 6 characters. Auth endpoints sit behind a stricter rate limiter than the rest of the API.

2.3 Session rules

- **Tokens** are JWTs carrying the user id, valid 30 days, kept in Expo SecureStore with the user record beside them.
- **A 401 on any authenticated request signs the app out** through one shared handler, so a dead session cannot leave a poller retrying forever.
- **Google accounts have no password.** They are flagged `isOAuthUser`, and Settings hides password change for them.

- **Sign-out clears everything per-account**: push token deregistered first (while the token still authorises it), then SecureStore, then every module cache and provider (section 8.5).

2.4 Onboarding

After verification the app opens Setup profile: avatar from the photo library, username and bio, with **Skip for now**. Skipping is a real path - the profile works empty, and Settings can fill it later. Push permission is requested once, on first sign-in, by the push registration hook; a device token is bound to one account at a time, so signing in as someone else on the same phone moves it over.

[Insert Screenshot: setup-profile screen with the avatar picker open]

2.5 Account management

Action	Where	Behaviour
Change bio, avatar, username	Settings	Writes straight back to the profile; the app updates its own copy from the response
Change password	Settings > Security	Hidden for Google accounts
Change language	Settings	12 languages; the choice is also reported to the server so push notifications arrive in it
Change theme	Settings	9 palettes
Blocked accounts	Settings > Your data	Section 8
Help & Feedback	Settings > About and support	Section 8
Delete account	Settings > danger zone	Confirmed twice, then the account and its content are removed

2.6 Terms acceptance

Agreement to the Terms of Service and Privacy Policy is collected at sign-up and recorded on the account.

- **Register screen** - a mandatory checkbox reading *I agree to the Terms of Service and Privacy Policy*, each document a tappable link that opens the public page in a browser. Register stays disabled until it is ticked.

- **Google sign-in** - the login screen carries the same line above the button, since tapping it is the agreement.
- **Server-side** - `POST /api/users/register` refuses anything where `acceptedTerms` is not exactly `true`, so the requirement does not rest on the app's own checkbox. Accounts store `termsAcceptedAt` and `termsVersion` (currently `2026-09-20`, from `backend/utils/legal.js`); Google accounts get both fields when they are created.
- Bump `TERMS_VERSION` when the terms change materially - it is what tells you who agreed to which text.

[Insert Screenshot: register screen with the terms checkbox, and the login screen's Google disclaimer]

3. The Media Feed (Discover)

The Home tab holds two pages side by side in a horizontal pager: **Media**, the social feed, and **Discover**, catalogue browsing. Media has no sort or type controls - recency, engagement and who you follow are blended into one ranking instead of being exposed as switches.

3.1 Ranking

Organic posts are scored per request:

```
score = 3 \cdot likes + 8 \cdot follows_{author} - 0.15 \cdot hours_{age}
```

A like is worth about 20 hours of freshness; following the author about 53 hours. That keeps a brand-new post from someone you don't follow visible, without letting an old popular post sit at the top forever.

Official posts bypass the score entirely. A flat boost was tried and failed - a fresh official post starts at zero likes and lost to anything already liked. Instead official posts take fixed absolute slots: the first post of the feed is always official, then one every 3 to 5 organic posts. The gap varies by a deterministic hash of the position, so it looks unpredictable but stays identical across refetches of the same page. This applies only to the **All** scope; **Following** is strictly the people you follow.

3.2 Scope switch

Scope	Contains
All	Everyone, ranked, with official posts at fixed slots

Scope	Contains
Following	Only people you follow, same ranking, no official inserts

Every fifth post the feed inserts a **suggested users** card, woven into the list rather than pinned to the top.

[Insert Screenshot: Media feed showing the All/Following switch and a suggested-users insert]

3.3 Post types

Seven types, each with its own card treatment and colour badge:

Type	Card shows
`review`	Poster, title, star rating, review text, full text opens in a modal
`recommendation`	Poster, title, optional note, add-to-watchlist action
`list`	Name, item count, a horizontal film strip of posters
`analysis`	A CineGraph persona card, opens the full analysis deck
`poll`	Question, options, live vote counts, one vote per person
`badge`	The earned badge's art, family, tier and count
`official`	Cinephix announcements, tips, trivia, on-this-day, stats; words inside the text can be tappable in-app links

[Insert Screenshot: a gallery of all seven post card types]

3.4 Interactions

- **Like** - optimistic, reversed if the server refuses. One notification per post however many likes it gets; a new like re-raises it, an unlike only corrects the count.
- **View** - counted once per person per post, feeding the ranking.
- **Vote** - polls only; your own vote is returned with the post so the card shows what you picked.
- **Share** - into a DM or a community as a rich card (section 5.6).

- **Report / block** - the three-dot button on anything that is not yours (section 8).
- **Open author** - avatar and username open the public profile; official accounts render their name in caps with a verified tick.

There is **no comments feature**. Reviews carry likes, and conversation happens in chats - worth knowing when reading anything that mentions comments, including the `Comment` entity type reserved in the report schema.

3.5 Above the feed

The Media page also carries the people podium (top followed/most active, with its own news modal), an online-friends bar, a currently-watching rail, and the compose button, which opens a menu of what you can post.

3.6 Tap handling (TapSafePressable)

On Android's new architecture, React Native's `Pressable` measures a button when the finger lands and cancels the press if the finger appears to leave that area. Those measurements go stale for a screen that sat underneath while the keyboard resized the window - after which the page still scrolls natively but **every tap is ignored**, permanently. It hit Socials, then Media, then Profile.

The fix is app-wide, not per screen:

- `src/components/common/TapSafePressable.tsx` re-exports gesture handler's `Pressable`, which lets Android decide what was tapped instead of measuring. Every tappable surface in the app imports it (150 files).
- `src/components/common/GestureModal.tsx` wraps React Native's `Modal` with its own `GestureHandlerRootView`. A Modal is a separate native window and inherits no gesture root, so without this nothing inside one would respond. All 30 modals use it.
- `src/utils/tapSafe.test.mts` fails the build if any screen imports `Pressable` or `Modal` from `react-native` again.

Two related adjustments came with it: nested sheets that used `stopPropagation` to swallow backdrop taps now rely on gesture handler giving the inner press priority, and two animated rows moved their animation to an inner `Animated.View` rather than animating the pressable itself.

4. Search, Watched List & Custom Lists

Search covers five kinds of thing, and everything a person watches lands in one of two collections: the **watched list** (seen, usually rated) or the **watchlist** (to watch). Custom lists are anything else they want to group.

4.1 Search

Five tabs, each with its own filters: **Movies**, **Series**, **Reviews**, **Lists**, **Users**.

- Only the selected tab is mounted. Switching tabs **resets the query and that tab's filters** on purpose - a stale filter still applied on a tab you just arrived at reads as a bug.
- The search bar's filter icon carries a dot when something is applied.
- Content filters open in a bottom sheet: genre, year range, minimum rating, sort.
- Results paginate as you scroll.

[Insert Screenshot: search with the Movies tab active and the filter sheet open]

4.2 Ratings

Ratings are stored 1-10 and **displayed as 5 stars everywhere** (`formatFiveScale`). Rating a title from any surface - a content page, a swipe card, a review form - writes one rating per person per title, and feeds the CineGraph genre maps and the Sync engine in section 7.

4.3 Watched list and watchlist

Collection	Field on the user	Meaning
Watched	`watchlist`	Already seen; carries the rating when there is one
Watchlist	`toWatchList`	Queued to watch

Both are tabs of their own in the bottom bar's hidden stack, reachable from the profile drawer, and both support search within the list, filters and sorting. Marking something watched removes it from the watchlist and clears it from **currently watching** if it was the active title.

Currently watching is a single explicit slot: one title at a time, set from a content page, cleared manually or when the same title is marked watched.

4.4 Custom lists

A list has a name, description, an ordered set of items, a privacy flag and likes.

Action	Where
Create	List editor - name, description, privacy, then add titles by search or suggestions

Action	Where
Edit	The list page's edit mode: rename, re-describe, reorder, remove items, **toggle Public/Private**
Reorder your lists	Drag on My Lists; the order persists per owner
Like	Any public list
Share	As a rich card into a chat (section 5.6)
Delete	Owner only, confirmed

Privacy is per list. A private list is visible only to its owner: the profile endpoint filters non-public lists out for every other viewer, and Discover only ever queries public ones. The one deliberate exception is the moderator view in section 9, which passes an explicit flag and labels private lists as private.

[Insert Screenshot: list detail in edit mode with the Public/Private toggle visible]

4.5 Finding other people's lists

- **Discover lists** - public lists, newest and popular, each card showing a swipeable film strip of up to 20 preview posters.
- **A profile's Lists page** - that person's public lists.
- **For this title** - which lists contain the title you're looking at, from its content page.

4.6 Content pages

Every title has a detail page reachable from anywhere it appears, carrying: poster and metadata, your rating, watched/watchlist actions, currently-watching, the full cast, similar titles, reviews (all, friends, popular), the lists it appears in, and a write-review screen. Because the same page is pushed from several stacks, it takes a `basePath` so it returns to where it was opened from.

5. Socials & Communities (The Chat Engine)

The Socials tab holds two pages behind a capsule switcher: **Messages** (the inbox) and **Communities**. Both are realtime over the same Socket.io connection, and both render from a cache on the first frame.

5.1 Socket protocol

```
sequenceDiagram
  participant App
  participant Socket
  participant DB
  App->>Socket: join-room {roomType, roomId}
  Socket->>DB: membership check
  Socket-->>App: error if not allowed
  App->>Socket: send-message {text, clientTempId}
  Socket->>DB: persist
  Socket-->>App: new-message (to the room)
  Socket-->>App: conversation-updated (to each member)
```

The connection authenticates with the same JWT. Membership is checked on join **and** on send, so a stale client cannot post into a room it was removed from, and blocked pairs cannot reach each other. `clientTempId` lets the sender reconcile its optimistic bubble with the stored message. Sockets disconnect when the app backgrounds and reconnect on foreground, refreshing the inbox on connect.

5.2 Messages (the inbox)

- DM rows and community rows in one list, newest activity first, each with avatar, last message preview and unread count.
- Blocked people's DMs are filtered client-side as well as server-side, so a row disappears the moment you block.
- In a community row, a preview written by someone you blocked is blanked while the row itself stays.
- **The Socials tab badge counts unread chats, not unread messages.** A community with 45 unread messages adds 1. The per-chat counts still show the real numbers inside the inbox.

[Insert Screenshot: Socials inbox showing the Messages/Communities switcher and the tab badge]

5.3 Communities

- Cards show the community's art, member count, hashtags and how many friends have joined.
- **Join and Leave live on the community's own page**, not on the cards - a deliberate call so browsing never mutates membership by accident. Leaving asks first, in the same confirm dialog style as account deletion.
- **Filter** (header popover): category and membership.

- **Sort** (header popover): Most Members, Friends Joined, Name A-Z, Name Z-A.
- Filters and sort compose, and the result is cached per owner.

[Insert Screenshot: Communities grid with the filter and sort popovers open]

5.4 Chat room

- Messages are held oldest-first, paginated upward, cached per room.
- **WhatsApp-style date separators** group the conversation by day: Today, Yesterday, the weekday for the last week, then a full date.
- Long-press a message to report it or block its sender - except on the official account, which cannot be either.
- The tab bar hides while the keyboard is open, so the composer sits on the keyboard.

5.5 Recommendations

From a content page you can recommend a title to someone. The flow picks recipients, attaches an optional note, and posts a rich card into the chat. A filter sheet narrows who you're sending to.

5.6 Premium share cards

Five rich card types share one presentation component (`PremiumChatCard`): a blurred backdrop drawn from the subject's own art, an eyebrow label with icon, a hero, a body and a call to action.

Card	Hero	Opens
Profile	Avatar in its tier frame, @username, followers, bio, top genres	That profile
List	Up to three posters fanned like cards in a hand, name, owner, count	That list
CineGraph analysis	A donut of the top three genres with their shares, the persona nickname	The analysis deck
Sync (CineMatch)	The compatibility score and its headline	The match analysis
Recommendation	Poster, title, year, the sender's note	That title

Older shares that predate a field simply show less rather than breaking - each card degrades to whatever the payload carries.

[Insert Screenshot: a chat thread showing all five premium card types and a date separator]

6. The Activity Footprint

The Activity page is a person's own record of what they've done in the app, written as sentences rather than log lines and grouped by day. It is reached from the profile drawer and from Settings.

6.1 What it covers

Eleven categories, drawn from several collections and merged into one timeline:

Category	Reads from	Example sentence
Ratings	`ratings`	You rated *Parasite* 4.5
Reviews	`reviews`	You reviewed *Dune*
Liked reviews	review likes	You liked a review of *Heat*
Liked lists	list likes	You liked the list Animations
Follows	`footprintevents`	You followed @ana
Created lists	`customlists`	You created a new list Noir
Watched & Watchlist	user collections	You added *Arrival* to your watchlist
Favorites	`favoriteevents`	You made *Alien* a favourite
Communities	`groups`	You joined Horror Club
Media posts	`shares`	You posted a review
CineMatch & CineGraph	analysis logs	You ran a Sync with @sam

6.2 Filters

Two dropdowns in the header:

- **Range** - the period covered.
- **Types** - a checklist. Every type is ticked by default; unticking one excludes it. An empty selection means *everything*, which is also what the server expects, so "all" costs no extra parameters.
- The filter button carries a badge showing how many types are currently hidden, so a narrowed view is never silent.

[Insert Screenshot: Activity page with the type checklist open and a hidden-count badge]

6.3 Presentation

- Entries are grouped under day headings, newest first, with each day's key derived from the local date - a fix for duplicate day headers when entries crossed a timezone boundary.
- Sentences are written in the second person and composed from i18n keys, so they translate as whole phrases rather than glued fragments.
- **"Media posts"** is the English label for what Turkish calls *Medya*. The product name differs per language deliberately: English says Media, Turkish keeps Medya, and a locale test enforces it.

6.4 Caching

Activity is cached per query (range plus categories) and per owner. Changing a filter re-reads from cache when it can. The whole cache is dropped when the signed-in user changes, and when someone is blocked, since their content must leave the timeline immediately.

7. Advanced Analytics (CineMatch & CineGraph)

Two engines sit on the same data. **CineGraph** describes one person's taste. **CineMatch (Sync)** compares two people. Both read from ratings and the per-genre maps kept on the user document.

7.1 CineGraph

Each user carries `movieCineGraph` and `seriesCineGraph`: a map of genre to `{ count, ratedCount, avgRating }`, maintained as they rate. From it the app derives:

- **The genre ring** - an animated donut, one arc per genre. Tapping a legend row focuses that genre: its arc thickens, the rest dim to 45%, and the centre and caption cross-fade to explain the share. Slice hit-testing is done by geometry on the ring band rather than per-slice touch targets, because per-slice handlers picked the wrong slice.
- **Per-genre detail pages** - what you rated in that genre, and how you rate it versus your overall average.
- **Top genres** - the three most-rated genres, reused on share cards and profile headers.
- **Recent taste** - the last 30 days against your all-time shares, with up/down chips.

[Insert Screenshot: CineGraph tab with the genre ring and one genre focused]

7.2 Profile analysis (the persona)

An on-demand pass over a profile that returns a persona nickname ("The Auteur"), a genre breakdown, recent and favourite titles, and stats. It calls Gemini when `GEMINI_API_KEY` is set and otherwise falls back to a deterministic local generator with the same shape. Results are kept in history, can be shared into a chat as a card, and open as a swipeable deck.

7.3 CineMatch (Sync) scoring

A hybrid model, blended per candidate:

Component	Weight	Method
Collaborative	35%	Pearson correlation over commonly rated titles; falls back to average-rating similarity when there is no overlap
Content-based	25%	Favourite-title overlap
Social	20%	Mutual following and followers
Genre	15%	Cosine similarity over genre counts
Bonus	5%	Activity, recency, trending

Weights shift when the viewer narrows the inputs: turning off average ratings moves the collaborative weight onto genre, and turning off counts moves genre onto collaborative, so the total always sums to 1.

Filters: movie or series, a genre subset, and whether to use counts, average ratings or both.

7.4 Sync surfaces

- **Candidates** - a ranked deck of people, swipeable, each card carrying the score and its headline reasons.
- **Score against one person** - shown at the top of their CineGraph tab, cached per person and per content type.
- **Analyze** - a written compatibility read for a pair, produced through the LLM adapter (Groq, OpenRouter, OpenAI or a mock, chosen by which key exists). One call per pair, cached per person, filters and language, so two taps do not produce two different texts.
- **Sync summary badge** - the animated result card at the end of the deck.

[Insert Screenshot: Sync candidate deck and a single match score card]

7.5 Badges

Badges are computed server-side from real counts, never trusted from the client, across several families: genre devotion (8 genres, two tiers), volume, ratings, reviews, lists, followers, influence, podium, Sync milestones, founding member, membership length, streaks, and seasonal ones. Earning a badge can be posted to the feed as a badge card, and a badge detail sheet explains what it took.

8. Trust, Safety & Support

Three systems: reporting content, blocking people, and writing to support. Blocking is enforced globally rather than screen by screen, which is what makes it trustworthy.

8.1 Blocking

A block is recorded on the blocker (`User.blockedUsers`) and works **both ways**: the two people become invisible to each other. The hidden set for a viewer is everyone they blocked plus everyone who blocked them.

```
flowchart TD
  A[Block tapped] --> B[Confirm dialog]
  B --> C[Server: add to blockedUsers]
  C --> D[Unfollow both ways<br/>+ drop follow records]
  D --> E[Invalidate both cached sets]
  C --> F[Device: purge caches<br/>chats, activity, profiles]
```

Enforcement happens in three places:

- **A global response filter.** Registered before every route, it strips list entries made by anyone hidden - by author keys (`user` , `owner` , `sender` , `actor` , `author` , `otherUser` , `targetUser`) or because the entry is that person. Feeds, search, leaderboards, member lists, inboxes and notifications are all covered by this one rule, **including endpoints written later**. Requests from people with no blocks are never even parsed.
- **Not-found guards.** Routes that return one thing by a person's id answer 404 rather than an empty page: profile, favourites, collections, ratings, CineGraph, genre detail, their posts, badges, every Sync route.
- **Explicit checks** in DM creation, sending and reading, socket room access, following, list and post detail.

A community keeps showing after you block one of its members; only that person's preview line is blanked, because hiding whole communities over one member was the wrong trade.

8.2 The blocked profile

Situation	What you see
They blocked you	"Profile not found"
You blocked them	Their picture and @username, "You blocked this user", and a prominent **Unblock** button

Unblocking restores the full profile in place. The server sends only name, picture and tier for that stub - nothing they have made. Settings also lists every blocked account with its own Unblock.

[Insert Screenshot: the "You blocked this user" profile state beside the Blocked accounts list]

8.3 Reporting

The three-dot button on a review, post, list or profile, and long-press in a chat. Six reasons: spam, harassment or hate speech, inappropriate content, spoiler without warning, impersonation, something else, plus an optional note.

- A report stores a **snapshot** of the reported thing, so moderators see what the reporter saw even if it was edited or deleted since.
- Reporting your own content is refused; 20 reports per hour per person.
- Reporting the same thing twice while the first is still pending returns quietly rather than queueing a duplicate (enforced by a partial unique index).
- Entity types: User, List, Review, Share, Message, and `Comment`, reserved for a feature that does not exist yet.

8.4 Official account protection

The official Cinephix account cannot be blocked or reported: the server refuses both with 403, and the app hides the three-dot button, the long-press menu and the profile's Report/Block row for it. The app learns which ids are official from a small endpoint at sign-in, so the buttons are hidden wherever that account appears without touching every populate in the API.

8.5 Help & Feedback (support tickets)

Settings > About and support > **Help & Feedback**, carrying a badge for unread replies.

- **Hub** - every ticket, newest first, with anything newly answered lifted to the top. Each card shows type, a colour-coded status (amber Open, blue In progress, green Resolved, grey Closed), title, date and a dot for a new reply.
- **New ticket** - four type cards (bug report, feature request, support, other), a title and details, with counters. Ten tickets per day per person.
- **The ticket** - what they wrote, then every reply from support, oldest first. Opening it clears the unread dot here and on the server.
- **Push** - answering or moving a ticket notifies the person in their own language, carrying the ticket id so tapping opens it. Repeat updates about one ticket collapse in the tray.

[Insert Screenshot: Help & Feedback hub, new-ticket form and an answered ticket]

8.6 Privacy fixes made during this work

Two leaks found and closed while building the above: the public profile endpoint was returning every user's **email address** to any signed-in caller (now admin-only), and their **block list and admin flag** (now removed from the response).

9. Web Admin Dashboard (The Command Center)

The moderator tools live in the Vite/React web app at `/admin`. English only - it is an internal tool. Everything it shows comes from `/api/admin`, which every request re-checks.

9.1 Access control

- Admin is granted **from the server only**: `node backend/scripts/setAdmin.js <email-or-username> on`. No endpoint can set it.
- The page re-reads the signed-in user from `/users/me` on every visit, because a stored copy could predate losing admin.
- Every `/api/admin` route sits behind `authMiddleware` + `requireAdmin`: anonymous requests get 401, signed-in non-admins get 403.
- `requireAdmin` also turns the block filter off, so a moderator who blocked someone still sees their content.
- Not signed in? The page shows its own sign-in form - the consumer web client is retired, so there is no other login route.

9.2 Reports queue (`/admin`)

- Pending, Resolved and Dismissed counts double as tabs; type chips (Profiles, Reviews, Media posts, Lists, Messages) carry their own pending counts. 20 per page.
- Each card: type, reason, status, age, the reported person and the reporter (both linking to their admin profile in a new tab), the snapshot taken when it was filed, the reporter's note, copyable ids, and who decided it with any note.
- Actions: **Resolve**, **Dismiss**, or **Reopen**, each with an optional note. Reopening clears the previous reviewer, timestamp and note.

[Insert Screenshot: reports queue with the status tabs and one expanded report card]

9.3 Support tickets (`/admin/tickets`)

A separate queue, since these are people asking for help rather than content to judge.

- Status counts as tabs (All, Open, In Progress, Resolved, Closed) plus type filters with counts.
- Queue on the left, the ticket being answered on the right.
- The panel shows the full description, the writer (picture, tier, **username linking to their admin profile**), and every reply so far.
- Actions: a status dropdown and a reply box. Saving either one notifies the person by push, and the panel says so.

[Insert Screenshot: ticket queue with a ticket open and the reply panel visible]

9.4 Admin profile view (`/admin/users/:id`)

The person as the app shows them, so a report can be judged in context rather than from a snapshot alone.

Main column - a desktop rebuild of the mobile public profile:

Tab	Contents
Posts	Every post type as the feed frames it, with likes, views and ids. Paged
Reviews	Everything they've written, read from the reviews collection - a reported review may never have been posted to the feed
Favorites	Poster grid with their own ratings
Lists	Preview posters, description, counts; **private lists included and labelled**

Tab	Contents
Watched list / Watchlist	The 60 most recent with titles, and the real total
CineGraph	Rated count and average rating per genre, movies and series side by side

The header carries picture, @username, tier, Official/Admin tags, follower counts, bio, top genres, plus moderator-only details: email, join date, user id, and an amber banner when reports are pending.

Right column - the moderation panel: counts by status, a switch between reports about them and reports they filed, and the same Resolve/Dismiss/Reopen actions without leaving the profile.

[Insert Screenshot: admin profile view with the moderation sidebar]

9.5 Running it locally

```
1. cd backend && npm run dev          # API on :5000
2. cd frontend && npm run dev         # web on :5173
3. node backend/scripts/setAdmin.js <your-username> on
4. open http://localhost:5173/admin
```

The dev server proxies `/api` to port 5000, so only port 5173 needs forwarding from a Codespace and there is no CORS setup. A Google-only account has no password and cannot sign in here.

9.6 Removing content and suspending accounts

The two actions that change the app itself. Both are red, both ask a second time before firing, and both are admin-only.

Delete content sits on any report of a review, post, list or message, and under each item on the profile view's Posts, Reviews and Lists tabs. It removes the item permanently and takes its loose ends with it:

Removing	Also removes
Post	Its like notifications
List	Activity entries for liking it, and any post showing it off
Message	Nothing else, but the chat's last-message preview is recomputed

Removing	Also removes
Review	Nothing else

The report keeps its own snapshot, so the record of what was removed survives the removal. Every pending report about that item is resolved, noted with who acted and why.

Suspend account sits in the profile header (with a required reason) and on any report. A suspended account keeps its data but loses access: the next authenticated request is refused with 401, and the app signs itself out and shows the reason on the login screen. Pending reports about them are resolved. Lifting a suspension restores access immediately.

Three accounts are out of reach: the official account, other moderators, and your own.

[Insert Screenshot: a report card showing Delete and Suspend, and the profile header's suspension banner]

Appendix A - API surface and data model

A.1 Route groups

Mount	Covers
`/api/users`	Accounts, auth, profiles, follows, favourites, collections, CineGraph, blocking, push tokens, suggestions, presence
`/api/users/me`	Watched list and watchlist writes
`/api/lists`	Custom lists: discover, popular, mine, CRUD, items, likes, reorder
`/api/movies`, `/api/series`, `/api/content`	Catalogue and per-title stats
`/api/ratings`, `/api/reviews`	Scores and reviews, including feeds and likes
`/api/match`	Sync candidates, pair scores, written analyses
`/api/feed`	Activity feed, leaderboards, podium
`/api/groups`, `/api/conversations`	Communities and DMs
`/api/shares`	Feed posts, likes, views, votes
`/api/notifications`, `/api/badges`	Notification feed, badge computation

Mount	Covers
`/api/reports`, `/api/tickets`	Filing reports, writing support tickets
`/api/admin`	Everything moderators use
`/api/waitlist`	Landing-page signups

A.2 Moderation and support endpoints

Method + path	Who	Does
`POST /api/users/:id/block`	Any member	Blocks, unfollows both ways; 403 for the official account
`DELETE /api/users/:id/block`	The blocker	Unblocks
`GET /api/users/me/blocked`	Any member	Who you blocked
`GET /api/users/official`	Any member	Official account ids
`POST /api/reports`	Any member	Files a report with a snapshot
`POST /api/tickets`	Any member	Opens a support ticket
`GET /api/tickets/me`	Any member	Their tickets plus an unread count
`PATCH /api/tickets/:id/read`	Owner	Clears the unread dot
`GET /api/admin/reports`	Admin	Queue, filtered by status, type, person
`GET /api/admin/reports/stats`	Admin	Queue sizes
`PATCH /api/admin/reports/:id`	Admin	Resolve, dismiss or reopen
`GET /api/admin/tickets`	Admin	Ticket queue
`PATCH /api/admin/tickets/:id`	Admin	Status and/or reply, then push

Method + path	Who	Does
`GET /api/admin/users/:id`	Admin	Summary, report counts, suspension state
`GET /api/admin/users/:id/{profile,cinegraph,favorites,collections,reviews,shares,ratings}`	Admin	The profile view's data
`DELETE /api/admin/content/:type/:id`	Admin	Permanently removes a review, post, list or message, with its cascades, and resolves the pending reports about it
`PATCH /api/admin/users/:id/suspension`	Admin	Suspends or restores an account; suspending needs a reason and closes pending reports about them

A.3 Ticket and report schemas

Field	Report	Ticket
Owner	`reporterId`	`userId`
Subject	`reportedEntityId` + `entityType` + `reportedUserId`	-
Kind	`reason` (6 values)	`type` (4 values)
Free text	`description`	`title` + `description`
State	`status`: pending, resolved, dismissed	`status`: Open, In Progress, Resolved, Closed
Evidence	`snapshot` of the content as filed	-
Moderator output	`reviewedBy`, `reviewedAt`, `resolutionNote`	`adminReply`, `replies[]`, `repliedAt`
Read state	-	`userReadAt`
Rate limit	20/hour	10/day

A.4 Push notification types

Type	Trigger	Opens
`follow`	Someone follows you	Their profile
`message`	A DM arrives	That chat
`share_like`	Your post is liked (collapsed, 5-minute cooldown)	That post
`ticket_update`	A moderator replies or changes status	That ticket

Push copy is translated into all 12 languages and a test fails if it drifts from the app's own wording.

Appendix B - Release readiness

B.1 Automated coverage

Suite	Tests	Command
Backend	296 across 20 files	`cd backend && npm test`
Mobile	193 across 18 files	`cd mobile && npm test`
Types	Clean	`cd mobile && npx tsc --noEmit -p .`
Mobile bundle	Builds	`cd mobile && npx expo export --platform android`
Web	Lint + build clean for admin and config	`cd frontend && npx vite build`

Tests are pure-logic by design: no database, no React renderer. Notable guards - the block filter against real response shapes, report and ticket validation, push copy matching the app's translations, locale coverage for every new string, and `tapSafe.test.mts`, which fails if any screen reintroduces React Native's `Pressable` or `Modal`.

B.2 Known gaps before submission

- [x] ~~**Terms acceptance at registration** - built (section 2.6): a mandatory checkbox, linked documents, and a server-side requirement recorded on the account.~~
- [x] ~~**Moderator content removal and user suspension** - built (section 9.6): moderators can delete a review, post, list or message and suspend an account from the dashboard.~~

- [ ] **Device testing of the app-wide tap fix**, especially inside modals (section 3.6).
- [ ] **Push notifications are live in this environment.** Answering a ticket notifies the real account; set `PUSH_NOTIFICATIONS=off` to test quietly.
- [ ] **Appeals have no route in the app.** A suspended person is signed out with the reason, but cannot reply to it - their support tickets need a session they no longer have. Worth deciding before launch.

B.3 Deliberate product decisions

Things that look like omissions and are not:

- **No comments feature.** Reviews carry likes; conversation happens in chats.
- **No "in common" filter** on public collections - removed on purpose.
- **No manual sort or type filter on the Media feed** - the ranking replaces them.
- **Media vs Medya**: English says Media, Turkish keeps Medya, enforced by a test.
- **Web app retired as a consumer client**; only the landing pages and `/admin` remain routed.
- **The 3-second auth cache** is a real staleness window, accepted because every mutation returns its own fresh value.

B.4 Operational runbook

Task	Command
Grant or revoke admin	`node backend/scripts/setAdmin.js <email-or-username> on` / `off`
Start API	`cd backend && npm run dev`
Start web	`cd frontend && npm run dev`
Start mobile (tunnelled)	`mobile/scripts/start-codespace.sh`
Disable push while testing	`PUSH_NOTIFICATIONS=off` in `backend/.env`
Point the app at a backend	`EXPO_PUBLIC_API_BASE_URL`, `EXPO_PUBLIC_SOCKET_URL`
Allow a web origin	`FRONTEND_URL` (comma-separated)

B.5 Security posture

- Helmet, CORS restricted to `FRONTEND_URL`, rate limiting on `/api/`, stricter limits on auth.

- Passwords hashed with bcrypt; JWT sessions; push tokens hidden from every ordinary query.
- **No source maps in the production web build.** The deployed bundle is minified with no `.map` files - verified against the live site. Anything that must stay secret belongs on the server, since any shipped bundle can be read.
- Admin is server-granted only, re-checked per request, and every admin route refuses non-admins.

B.6 Version baseline

This document describes the tree at commit `3e38ef1` plus the uncommitted work on `furkan2_temp`: the report and block system, official-account protection, the blocked-profile flow, the support ticket system, the admin dashboard with its profile view, moderator content removal and account suspension, terms acceptance at sign-up, the app-wide tap fix, and the unread-chats badge. Use it as the baseline for patch notes; each future change should name the section it edits.